Ilya Romanovich

Backend & Embedded Systems Engineer

Minsk, Belarus · ilya.business.contact77@gmail.com · github.com/mdedz · LinkedIn

Summary

Backend and embedded systems engineer who works below the API layer as comfortably as above it. Designed and shipped a real-time device-telemetry platform spanning embedded C (ESP32/FreeRTOS), a Python/FastAPI backend, and a React operator console — including a cross-language Ed25519 device-identity protocol implemented independently in both Python and embedded C. Separately built and operated a fleet-management platform for Android-TV hardware combining low-level ADB/GPIO reverse engineering with a Django/Channels server doing signed real-time WebSocket RPC and secure over-the-air updates. Comfortable owning a system end to end: protocol design, backend, frontend, deployment.

Technical Skills

- **Backend:** Python, FastAPI, Django, Django REST Framework, SQLAlchemy, Alembic, PostgreSQL, Redis, Celery
- **Systems & Embedded:** C, C++, Rust, ESP-IDF/FreeRTOS, UART/RS-232 serial protocols, CRC16, GPIO, ADB, systemd
- **Security & Auth:** JWT, OAuth2, Ed25519 / applied cryptography, HMAC-signed messaging, TLS
- **Frontend:** React 19, TypeScript, Vite, Redux Toolkit, TanStack Query, Tailwind CSS
- **Infrastructure:** Docker, Docker Compose, GitHub Actions CI, Linux, Nginx
- **AI/LLM integration:** OpenAI / Azure OpenAI / Groq SDKs, function/tool-calling, structured-output validation, rate limiting

Experience

Independent Software Engineer **2020 – Present**
Self-Employed, Remote

Design, build, and operate backend and embedded systems for casino-industry telemetry, Android-TV device fleets, and applied-AI automation, working directly with clients on end-to-end delivery – protocol design through deployment.

- Designed a multi-tenant FastAPI backend and a matching zero-touch device-enrollment protocol (Ed25519 proof-of-possession, distinct short-lived machine-identity JWTs) for a casino slot-machine telemetry platform, then re-implemented the collector firmware from scratch in embedded C for an ESP32 target to prove the protocol was genuinely hardware-agnostic.
- Reverse-engineered per-chipset Android-TV control internals (ADB input-service class paths, DPAD navigation timing) and built a GPIO hardware relay fallback and self-healing watchdog for a remotely-managed device fleet, paired with a Django/Channels server providing signed, correlated WebSocket RPC and secure GitHub-App-based OTA updates.
- Built a real-time computer-vision tracking system (multi-threaded C++, NPU-accelerated inference) with two independent statistical calibration/drift validators to catch silent camera and lighting drift that would otherwise silently corrupt output.
- Delivered CI-gated, fully tested Django/DRF backend services (booking systems, caching/rate-limited API integrations) to technical-assignment standard, and a self-hosted multi-service IaaS control plane (Rust hypervisor agent + Django backend + React frontend) as a full-stack systems exercise.
- Built and shipped a CLI automation tool with real LLM-reliability engineering: truncation-detection-and-recovery, fail-open classification handling, and provider-agnostic LLM integration, backed by CI and a full test/lint pipeline.

Flagship Projects

SAS Casino-Protocol & Telemetry Platform *Python (FastAPI) · embedded C (ESP32/FreeRTOS) · TypeScript/React · PostgreSQL*

Designed a multi-tenant backend and operator dashboard for real-time casino slot-machine telemetry, with a cross-language Ed25519 device-identity protocol (proof-of-possession enrollment, short-lived machine JWTs) implemented independently in Python and hand-vendored embedded C. Solved 9-bit UART wakeup emulation over 8-bit hardware

UARTs via MARK/SPACE parity switching to communicate with real slot-machine cabinets. Backed by a 16-file / 3,000+ line test suite including full lifecycle end-to-end coverage.

Android-TV Fleet Control & Digital Signage Platform *Python (Django/Channels/ASGI) · Flask · WebSockets*

Built a remote device-management platform for Android-TV hardware: a signed, correlated WebSocket RPC protocol bridging Django Channels' async model back into synchronous REST views; a secure OTA update mechanism using short-lived GitHub App installation tokens so field devices never hold long-lived credentials; a device agent doing real ADB/GPIO reverse engineering with layered failure recovery (software retry – hardware relay – host-level watchdog reboot).

Real-Time Computer Vision Tracking System *C++ · OpenCV · RKNN (NPU)*

Built a multi-threaded, NPU-accelerated real-time object-tracking pipeline with two independently-designed statistical validators – one gating calibration quality, one detecting runtime drift via proper circular statistics and hysteresis state machines – to guard against silent sensor/environment drift that would otherwise corrupt output with no visible symptom.

Self-Hosted IaaS Control Plane *Rust (Axum/bollard) · Django/DRF · React/Redux · Docker*

Built a full-stack, self-hosted infrastructure-management platform: a Rust hypervisor agent doing real loop-device disk lifecycle management (create/format/mount/resize ext4 images) and Docker container orchestration, a Django/Celery/Redis backend with quota-based multi-tenancy, and a React/Redux frontend with live resource-usage polling.

AI-Powered Job Search Automation *Python · Groq/OpenAI-compatible LLM APIs*

Built a CI-tested CLI tool automating job-board interaction with real LLM-reliability engineering: a working truncation-detection-and-continuation protocol, fail-open handling of ambiguous LLM classification output, and a provider-agnostic LLM abstraction – full lint/type-check discipline (ruff, mypy `--strict`, pre-commit) and GitHub Actions CI.

Django REST API Services *Python · Django REST Framework · PostgreSQL · Redis*

Delivered two CI-gated backend services to technical-assignment standard: a booking system with correct interval-overlap validation, and a caching/rate-limited external-API integration with a documented health-check endpoint – both fully tested (9+ test files each) and Docker-deployable.

Education

Belarusian State Academy of Communications (BSAC) – coursework in progress (not completed)